

What's Actually Inside A Model File?



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

What IS a model, physically? **A big pile of numbers**

A trained model is two things: a **set of numbers** (the weights) and a **description of how they're arranged** (architecture).

TENSORS (the weights)

```
layer_0.attention.query.weight  
→ [[0.021, -0.114, 0.87, ...],  
    [0.334, 0.512, -0.22, ...]]  
shape: [4096, 4096] dtype: fp16
```

...repeated for every layer

METADATA

tensor names, shapes, dtypes, byte offsets

A tensor is just a multi-dimensional grid of numbers. $[4096, 4096] = \sim 16.8$ million values in one layer.

THE MATH BEHIND FILE SIZES

$7,000,000,000 \text{ params} \times 2 \text{ bytes (fp16)} = \sim 14 \text{ GB}$



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

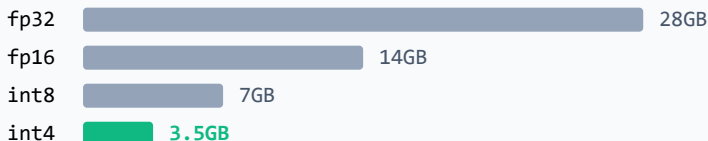
 Repost

Precision: Why the same model has different sizes

The dtype decides how many bytes each number costs. **Quantization** converts to lower precision, shrinking the file.

Precision	Cost	7B Size	Value Example
fp32	4 bytes	28 GB	0.1847264719
fp16	2 bytes	14 GB	0.18472
int8	1 byte	7 GB	0.184
int4	0.5 b	3.5 GB	0.19

SAME MODEL, 4 FILE SIZES



AI Coach John



Follow

↻ Repost

The Original Format (.bin): Where it started, and the problem

PyTorch originally saved models with Python's pickle, producing .bin or .pt files.

```
import torch
torch.save(model.state_dict(), "model.bin")
torch.load("model.bin") # ⚠ executes code
```

The critical flaw: Pickle stores instructions to reconstruct objects, including arbitrary code execution. Loading an untrusted .bin is like running a stranger's script.

THE SECURITY RISK

model.bin (pickle)

✓ Weights

⚠ Arbitrary Python code

torch.load() → CODE RUNS



AI Coach John



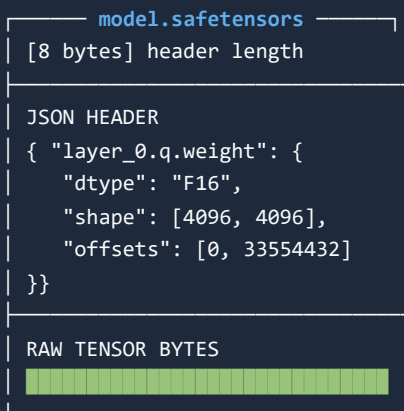
PROFITBRIDGE
Strive For Better Future

Follow

 Repost

Safetensors: The safe default

Safetensors stores a simple JSON header describing the data, followed by raw bytes. No code execution is possible.



- ✓ **Safe** — Data only, no executable code
- ✓ **Lazy loading** — Offsets let you read 1 tensor instantly
- ✓ **Fast** — Memory mapping (OS maps file to memory)



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

🔄 Repost

What "Memory Mapping" Actually Means: Why it's so fast

TRADITIONAL LOADING (pickle)

disk —copy—> RAM —deserialize—> Python objects —> GPU
└ the whole file, all at once, before usable ┘

MEMORY-MAPPED LOADING (safetensors)

disk ← OS maps file into address space → used
└ pages loaded on demand, no upfront copy ┘

Loading starts almost immediately. Multiple processes can share the mapped file without duplicating memory, and you load only what you need.



The gap between waiting a minute vs waiting a few seconds every time you spin up a model.



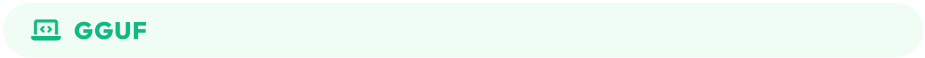
AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost



GGUF: Built for running locally on CPU

GGUF (successor to GGML) is designed for running quantized models efficiently on consumer hardware (llama.cpp, Ollama).

```
model.gguf
```

```
METADATA  
architecture: llama  
context_length: 8192  
quantization: Q4_K_M  
tokenizer: [vocab + merges] ★  
chat template ★
```

```
QUANTIZED TENSORS
```

★ = bundled in ONE file

★ = bundled in ONE file

Self-contained: Safetensors needs config files and tokenizers. GGUF packs it all into one artifact that just runs.

SAFETENSORS <pre> model.safetensors config.json tokenizer.json tokenizer_config.json generation_config.json</pre> (5 files)	GGUF <pre>└─ model.gguf</pre> (1 file, runs instantly)
--	---

```
├── model.safetensors
├── config.json
├── tokenizer.json
├── tokenizer_config.json
└── generation_config.json
```

(5 files)

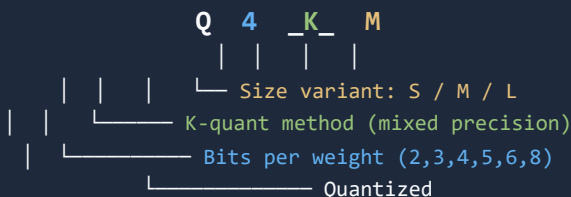
GGUF

- └─ model.gguf

(1 file, runs instantly)



Decoding GGUF Labels: What is Q4_K_M?



K-quants keep important weights at high precision and compress less important ones harder—better quality at the same size.

Q8_0	~7.2GB	Near-lossless, rarely needed
Q6_K	~5.5GB	Very high quality
Q4_K_M	~4.1GB	★ The usual sweet spot
Q3_K_M	~3.3GB	Noticeable degradation
Q2_K	~2.8GB	Significant quality loss



AI Coach John



PROFITBRIDGE
Strive For Better Future

Follow

↻ Repost

ONNX: Built for portability

ONNX (Open Neural Network Exchange) runs the same model across different languages and devices.



It stores the **computation graph**, not just weights. Any ONNX runtime can execute the operations (matmul, softmax) without PyTorch.

safetensors: "Here are the weights" (you supply architecture)

ONNX: "Here are the weights AND the ops to run"



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

All Four Compared: Which format for which job

	.bin (pickle)	Safetensors	GGUF	ONNX
Contains	Weights	Weights	Weights+tok+config	Weights+compute
Safe?	X Code risk	✓ Data only	✓ Data only	✓ Data only
Load Speed	Slow	Fast (mmap)	Fast	Moderate
Quantization	External	Supported	Built-in	Supported
Self-contained	No	No	✓ Yes	Mostly
Target	Legacy PyTorch	GPU training	CPU/local	Cross-platform
Used by	Older repos	vLLM, HuggingFace	Ollama, llama.cpp	Mobile, Web



Which Format Do You Need?

Follow the flow

Running locally on a laptop / CPU / Ollama?

↳ **GGUF** (Q4_K_M is the usual start)

Fine-tuning or serving on a GPU with vLLM?

↳ **Safetensors**

Deploying to mobile, browser, or edge?

↳ **ONNX**

Downloading from an untrusted source?

↳ Prefer safetensors/GGUF. **Avoid .bin**

Found only a .bin from a trusted source?

↳ Convert it to safetensors first



GGUF for local, Safetensors for GPU, ONNX for cross-platform. And never load an untrusted pickle.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

Sharded Files: Why big models come in pieces

Large models are split across multiple files, with an index mapping tensors to shards.

```
| model-00001-of-00004.safetensors    (~5 GB)
| model-00002-of-00004.safetensors    (~5 GB)
| model-00003-of-00004.safetensors    (~5 GB)
| model-00004-of-00004.safetensors    (~3 GB)
└─ model.safetensors.index.json ← the map
```

index.json:

```
{ "weight_map": {
  "layer_0.q.weight": "model-00001...",
  "layer_45.k.weight": "model-00003..."
}}
```

- ❗ Interrupted downloads resume per-shard.
- ❗ Filesystems/hosts often cap individual file sizes.
- ❗ Shards can load in parallel across multiple GPUs.



Working With Formats: Loading and converting

```
# --- Load safetensors directly ---
from safetensors.torch import load_file
weights = load_file("model.safetensors")

# --- Inspect WITHOUT loading to memory ---
from safetensors import safe_open
with safe_open("model.safetensors", framework="pt") as f:
    print(f.keys())          # tensor names
    t = f.get_tensor("layer_0.q.weight") # load just one

# --- Force safetensors from HF Hub ---
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained(
    "org/model",
    use_safetensors=True      # refuse pickle files
)

# --- Export to ONNX ---
torch.onnx.export(model, dummy, "model.onnx")
```



⚠ CONSEQUENCES

Why Format Matters: The practical impact

🔒 Security

Untrusted pickles can execute arbitrary code. Safetensors fixes this.

⚡ Startup latency

Memory-mapping starts serving in seconds instead of minutes.

⚙ Hardware fit

GGUF Q4 fits on laptops; fp16 safetensors needs datacenter GPUs.

📱 Deployment reach

ONNX is often the only path to mobile, web browsers, or embedded.

📁 Operational simplicity

GGUF's single-file design means one artifact to move and cache.

💾 Storage and bandwidth

Quantized formats cut download sizes by 4-8×, vital at fleet scale.



AI Coach John



PROFITBRIDGE
Strive For Better Future

Follow

↻ Repost



It's numbers in a file.

The format just decides what you can do with them.

SAFETENSORS → safe + fast (GPU)

GGUF → self-contained
(local)

ONNX → portable (edge/web)



AI Coach John



Follow

↻ Repost